Analyzing Running Times Of **Recursive Program** Code

Techniques of Analyzing Recursive Code

- Back Substitution Method
- Recursive Tree Method
- Master's Theorem

$O(n^2)$

```
demo(int n)
{
 if(n>0)
   {
      for(i=1; i<=n; i++)
          print (“message”);
      demo(n-1);
   }
else
return 1;
}
```

O(n logn)

```
demo(int n)
{
 if(n>0)
   {
      for(i=1; i<=n; i=i*2)
          print (“message”);
      demo(n-1);
   }
else
return 1;
}
```

$O(2^n)$

```
demo(int n)
{
 if(n>0)
   {
      print ("message");
      demo(n-1);
      demo(n-1);
   }
else
return 1;
}
```

O(logn)

```
demo(int n)
{
 if(n>1)
   {
      print (“message”);
      demo(n/2);
     }
else
return 1;
}
```

O(n)

```
demo(int n)
{
 if(n>1)
   {
          for(i=1; i<=n; i=i+1)
          print (“message”);
         demo(n/2);
     }
else
return 1;
}
```

O(n logn)

```
demo(int n)
{
 if(n>1)
   {
          for(i=1; i<=n; i=i+1)
          print (“message”);
        demo(n/2);
          demo(n/2);
    }
else
return 1;
}
```

O(n)

$$T(n) = c + 2T(n/2) \quad n>1$$

$$= c \quad n=1$$

Amount of work done varies at different levels

(max. work done at root)

$T(n) = 2T(n/2) + cn^2$ [n^2]

(work is done equally at all levels)

$T(n) = 2T(n/2) + cn$ [$n \log n$]

(max. work done at leaves)

$T(n) = 2T(n/2) + c$ [n]

Results

$T(n) = c + T(n-1) \Rightarrow O(n)$

$= 1 + T(n-1) \Rightarrow O(n)$

$= n + T(n-1) \Rightarrow O(n^2)$

$= n^2 + T(n-1) \Rightarrow O(n^3)$

$= \log n + T(n-1) \Rightarrow O(n \log n)$

$T(n) = 2T(n-1) + c \Rightarrow O(2^n)$

$= 3T(n-1) + 1 \Rightarrow O(3^n)$

$= 2T(n-1) + n \Rightarrow O(n2^n)$

$= 2T(n-1) + \log n \Rightarrow O(\log n.2^n)$

- $T(n) = T(n/2) + 1 = O(\log n)$

  $= T(n/2) + n = O(n)$

  $= 2T(n/2) + n = O(n \log n)$

Related contents can be found

Analyzing Running Time of Iterative Algorithms

https://cexpertvision.com/analyzing-running-time-of-iterative-algorithms/

Back Substitution Method

https://cexpertvision.com/back-substitution-method/

Video Lecture of the topic can be found at

Analyzing Running Time of Iterative Program Code

https://youtu.be/zcPF4ZXt5FI

Analyzing Running Time of Recursive Code using Back Substitution method

https://youtu.be/9OXy5V7jMYc

Analyzing Running Time of Recursive Code using Recursive tree Method

https://youtu.be/KPtjTsAmDn4